SLOVENSKÁ TECHNICKÁ
UNIVERZITA V BRATISLAVE
FAKULTA INFORMATIKY
A INFORMAČNÝCH TECHNOLÓGIÍ

Aglomeratívne zhlukovanie (2.c)

Meno: Juraj Černička

ID: 127128

Rok: 2024/25

Cvičenie: Ing. Martin Komák, PhD., Štvrtok 16:00

Zadanie:

V 2D priestore s rozmermi od +5000 do -5000 na oboch osiach náhodne vygenerujeme 20 bodov. Následne vygenerujeme ďalších 20000 bodov nasledujúcim spôsobom: Zoberieme už vygenerovaný bod a pridáme k nemu náhodný offset od +100 do -100 pre obe súradnice. To znamená, že vyberáme z bodov vygenerovaných aj prvým aj druhým spôsobom (takže dokážeme zobrať vygenerovaný bod, pridať offset, a potom zobrať tento nový bod, pridať offset a pridať tak tretí bod). Nový bod sa nesmie nachádzať mimo vyhradeného 2D priestoru.

Náhodná generácia bodov slúži na vytvorenie dát, ktoré sú vhodné pre klastrovanie (keďže týmto spôsobom vzniknú body v hromadných skupinách, kde jednotlivé skupiny sú od seba náhodne vzdialené).

Druhá časť zadania sa týka samotného klastrovacieho algoritmu. Používame dva rozdielne ale veľmi podobné algoritmy: Algomeratívne zhlukovanie, kde stred je centroid, a algomeratívne zhlukovanie, kde stred je medoid. Žiadny klaster nemá mať priemernú vzdialenosť bodov od stredu viac ako 500.

Pri algoritme máme použiť matice vzdialeností na zrýchlenie výpočtov, ako aj kompilátor PyPy.

Posledná časť zadania je vizualizácia dát. Každý klaster má byť vizualizovaný inak, napríklad inou farbou.

Reprezentácia údajov:

Jednotlivé body v 2D priestore sú reprezentované dvoma súradnicami: x a y. Tieto body sú uložené v liste, ktorý sa nachádza v triede Cluster_Map. Táto trieda taktiež obsahuje list klastrov. List bodov v priestore a list klastrov sú na začiatku behu programu hodnotovo ekvivalentné.

Ďalšou položkou tejto triedy sú matice vzdialeností. Nachádza sa tu matica vzdialeností medzi bodmi, a matica vzdialeností medzi klastrami. Obe matice sú na začiatku behu programu taktiež hodnotovo ekvivalentné.

Každý klaster je reprezentovaný inštanciou triedy Cluster. Táto trieda obsahuje funkcie na zmenu klastra (špecificky zmenu klastra po pridaní nového bodu) a stred daného klastra. Stred môže byť reprezentovaný podľa zadania buď centroidom, alebo medoidom. Informácia o tom, ktorý druh reprezentácie je aktuálny, je uložená v Cluster triede v premennej centre_is_centroid.

Poslednou triedou je trieda Cluster_Divisor, ktorá obsahuje funkcie slúžiace na vykonanie samotného algomeratívneho algoritmu, ako aj statickú funkciu na výpočet vzdialenosti dvoch bodov.

Algoritmus a jeho implementácia:

Na zgrupenie bodov do jednotlivých klastrov sme použili algomeratívne zhlukovanie. Tento algoritmus funguje nasledovne:

1. Na začiatku sa každý bod považuje za jeden klaster. Súradnice daného bodu sú zároveň aj stredom klastra (nezávisle od toho, či je použitý centroid alebo medoid na reprezentáciu stredu klastrov).
2. Nájdu sa najbližšie dva klastre a tie sa spoja dohromady. To znamená, že všetky body z klastra A aj z klastra B sa spoja do nového klastra C, a klastre A a B sa vymažú. V tomto kroku sa vypočíta aj nový stred klastra.
3. Krok dva sa opakuje, dokým by spojenie dvoch najbližších klastrov nespôsobilo, že priemerná vzdialenosť bodov vo vnútri klastra od jeho stredu presiahne hodnotu 500. Vtedy sa algoritmus ukončuje a nasleduje vykreslenie 2D priestoru s klastrami.

Na zrýchlenie výpočtov použijeme maticu vzdialeností jednotlivých klastrov. Indexy matice korešpondujú s indexami listu klastrov v triede Cluster_Map, čo dovoľuje jednoduchú kontrolu vzdialenosti akéhokoľvek klastra od ktoréhokoľvek iného použitím ich indexov a matice.

Po spojení dvoch klastrov sa jeden z klastrov zmení na novo vytvorený klaster, zatiaľ čo druhý sa vymaže. V matici sa teda vymaže celý jeden riadok s indexom vymazaného klastra. Vymaže sa tiež stĺpec v matici, zmazaním hodnoty s daným indexom v každom riadku matice. Nakoniec sa tento klaster zmaže aj v liste klastrov, čím sa zabezpečí, že indexy klastrov v liste klastrov budú stále priamo namapované na indexy matice vzdialeností. Nakoniec sa vypočíta nový stred klastra.

Pre výpočet centroidu sa spočítajú súradnice všetkých klastrov a predelia počtom klastrov. Táto operácia je jednoduchá, keďže každý klaster obsahuje list všetkých jeho bodov.

V prípade medoidu je to trocha zložitejšie, pretože potrebujeme poznať vzájomné vzdialenosti každého z bodov. Na to použijeme maticu vzdialeností všetkých bodov (ktorá je iná ako matica vzdialeností klastrov, lebo sa od počiatočného vzniku nikdy nezmení).

V klastri budeme mať uložený list súčtu vzdialeností každého bodu od všetkých ostatných bodov v klastri. Indexy tohto listu sa priamo mapujú na indexy listu všetkých bodov v klastri. Pri pridaní nových bodov pridáme nové vzdialenosti do tohto listu a k predošlým vzdialenostiam pridáme vzdialenosti daných bodov od novo pridaných bodov.

Algoritmus sa opakuje kým by spojenie dvoch najbližších klastrov nespôsobilo presiahnutie limitu priemernej vzdialenosti bodov od stredu v novo vytvorenom klastri. To priebežne kontrolujeme tak, že pred spojením dvoch klastrov pokaždé vytvoríme deep copy klustrov, ktoré chceme spojiť dohromady. Potom spojíme dohromady tieto kópie a vypočítame priemernú vzdialenosť bodov nového klastra od stredu. Na tvorbu deep copy klastrov používame knižnicu copy.

Vizualizácia bodov je dosiahnutá pomocou knižnice matplotlib. Body sú vizualizované inou farbou podľa toho, do ktorého klastra zapadajú. Stredy klastrov sú reprezentované červenými bodkami s čienym okrajom.

Ďalšie použité knižnice sú random (na generáciu náhodných počiatočných bodov), math (na výpočet vzdialenosti dvoch bodov podľa pytagorovej vety, presnejšie na výpočet odmocniny) a knižnica copy.

Časová zložitosť:

Vďaka použitiu matice vzdialeností všetkých bodov a listu vzdialeností pre každý bod v klastri je operácia výpočtu medoidu relatívne rýchla, a rozdiel medzi dĺžkou programu pre daný počet vygenerovaných bodov s použitím centroidov je porovnateľná s dĺžkou programu pre ten istý počet vygenerovaných bodov s použitím medoidov.

Výpočet matice vzdialeností má kvadratickú zložitosť, a pri 20000 bodoch sa generuje približne 20 sekúnd. Najdlhšie trvá samotný algomeratívny algoritmus, presnejšie prehľadávanie matice vzdialeností klastrov. Pri 20000 bodoch je dĺžka algoritmu medzi 45-50 minútami (aj pre centroidy, aj pre medoidy), kde veľkú väčšinu času zaberá prehľadávanie matice. Toto prehľadávanie má kvadratickú časovú zložitosť závislú od počtu počiatočných bodov.

Postupné zmenšovanie matice po každom spojení dvoch klastrov zrýchľuje algoritmus, ale nepredstavuje dostatočné zrýchlenie na prenesenie dĺžky prehľadávania mimo kvadratickú zložitosť.

Čas vykresľovania 2D priestoru s klastrami značne neovplyvňuje celkovú dĺžku bežania programu.

Na značné zrýchlenie programu sme použili just in time kompilátor PyPy.

Používateľská príručka:

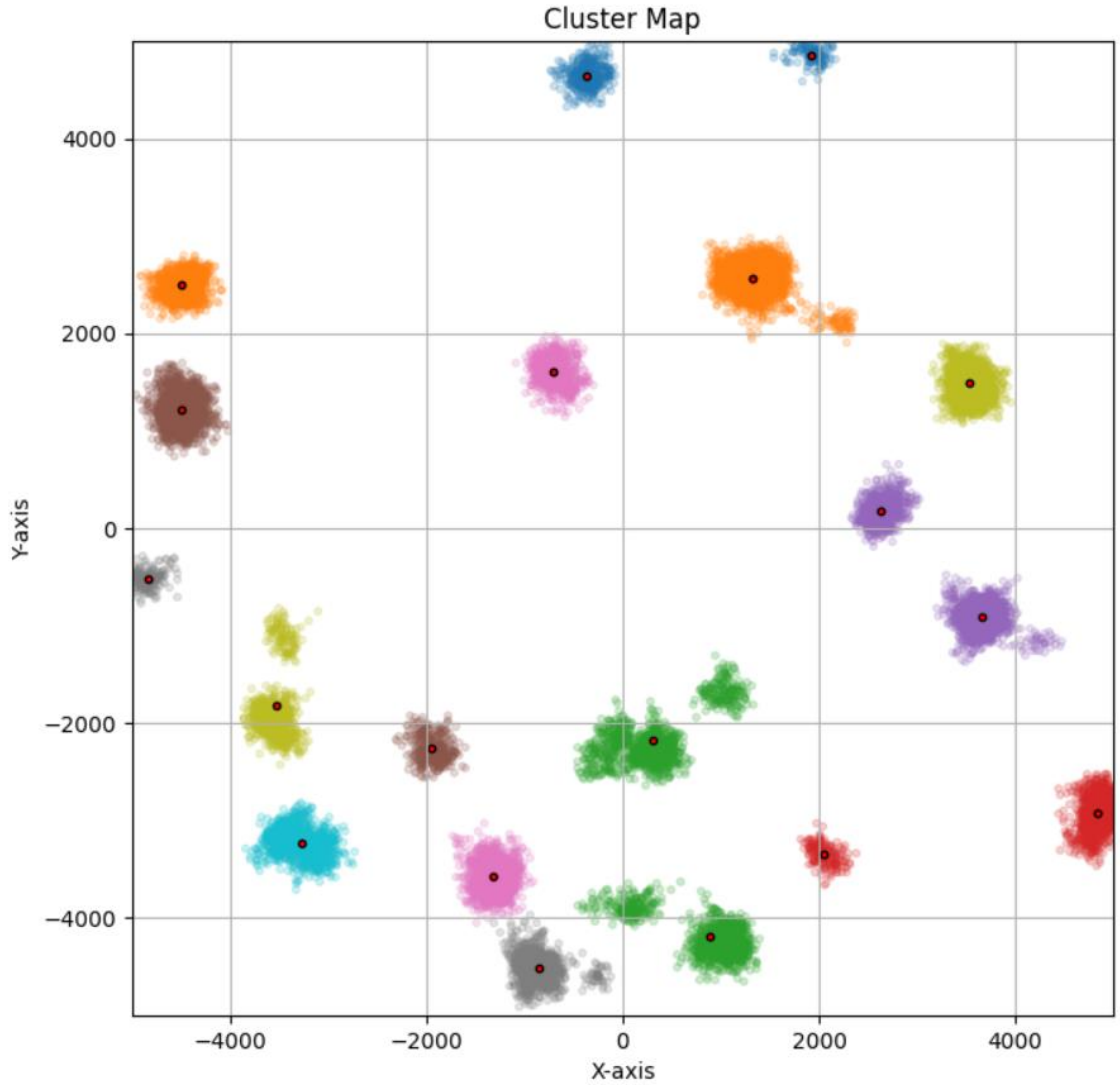
Po spustení programu (celý program je v jednom python súbore s názvom Clustering.py) sa do konzoly napíše „Should centre be centroid or medoid? (input '1' for centroid and '2' for medoid)“. Po špecifikovaní zapísaním danej hodnoty do konzoly sa spustí beh programu. V konzoli sú v tejto fáze vypisované časy priebehu funkcií, špecificky časy priebehu výpočtu matice vzdialeností, inicializácie inštancie Cluster_Map, a čas priebehu funkcie divide, ktorá vykonáva samotný algoritmus algomeratívneho zhlukovania. Po prebehnutí všetkých funkcií sa zobrazí vizualizácia klastrov v grafe generovanom knižnicou matplotlib.

Globálna premenná menom TOTAL_NUMBER_OF_POINTS zadáva celkový počet bodov v grafe (včetne 20 počiatočných bodov).

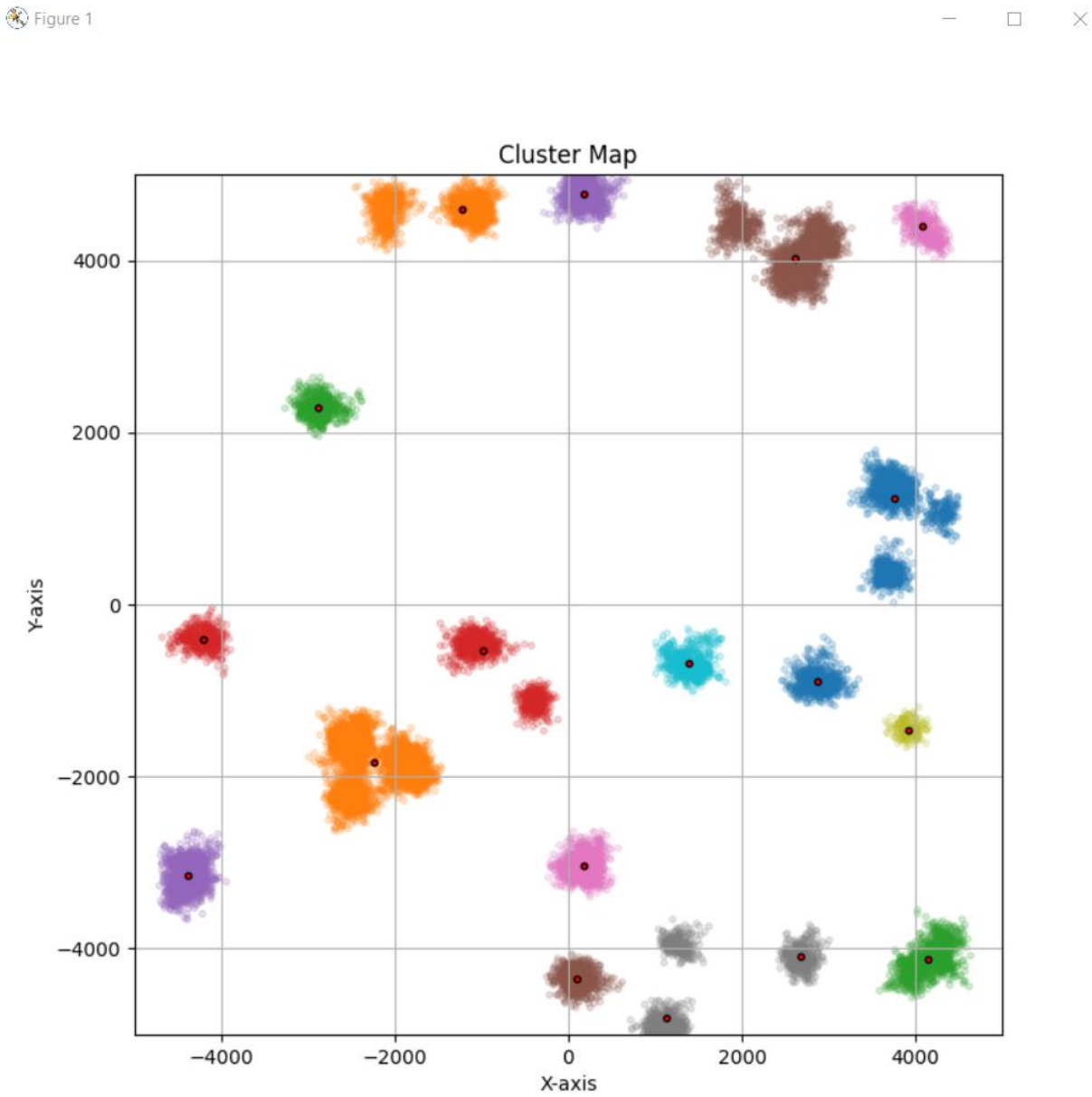
Vývoj riešenia:

Program nemal veľa výrazných vývojových miľníkov. Po počiatočnom dokončení algoritmu pre centroidy bol najväčším problémom pomalý priebeh programu. Ten bol vyriešený po stiahnutí a implementovaní kompilátora PyPy. Pre veľké množstvá bodov stále funguje celkom pomaly, čo som chcel vyriešiť použitím heapq, ale rozhodol som sa proti tomu pretože v zadaní bolo špecificky spomenuté použitie matice. Nakoniec som pridal funkciu pre počítanie medoidov a mechanizmus pre výber medzi medoidami a centroidami.

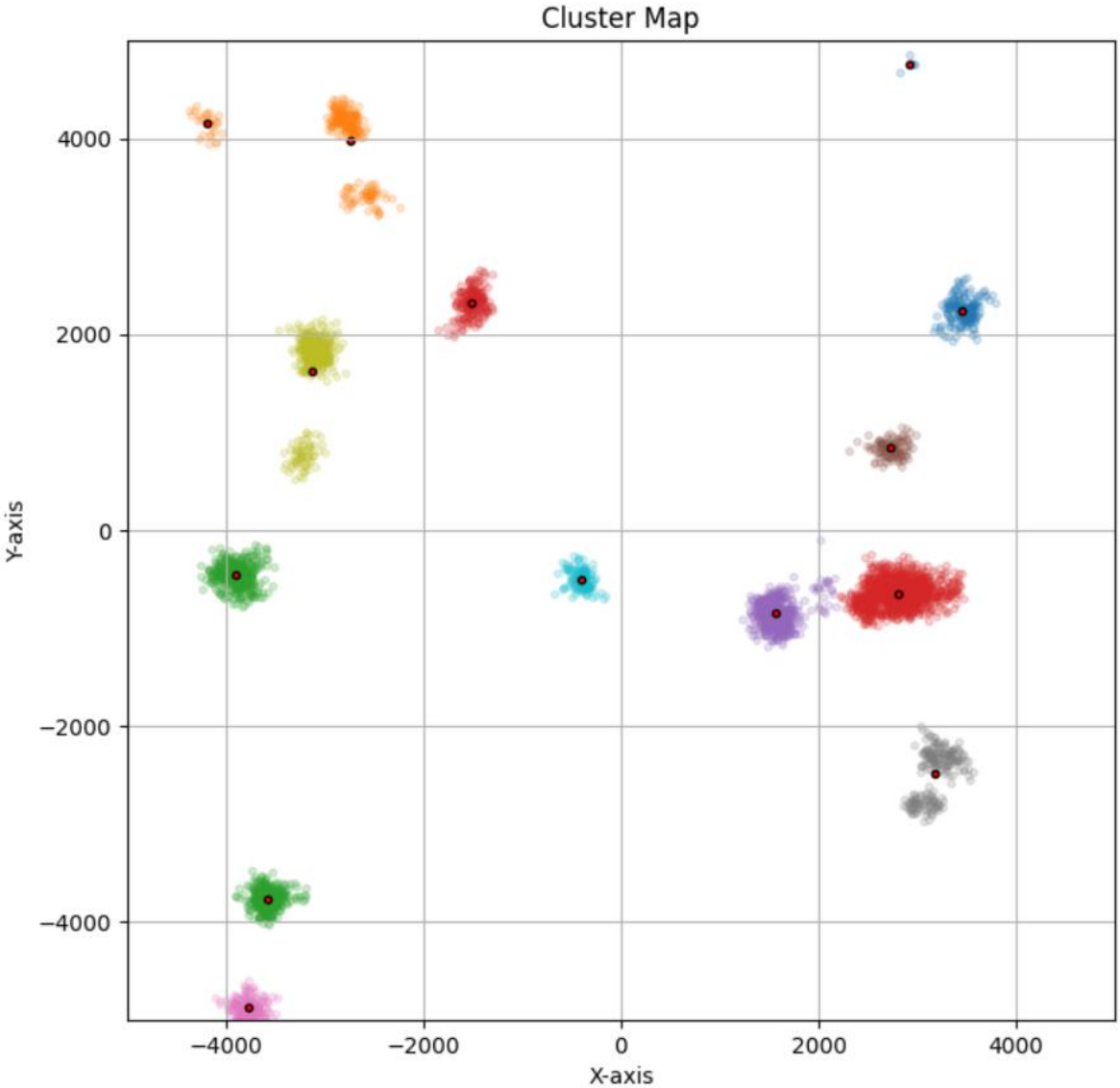
Vizualizácia:



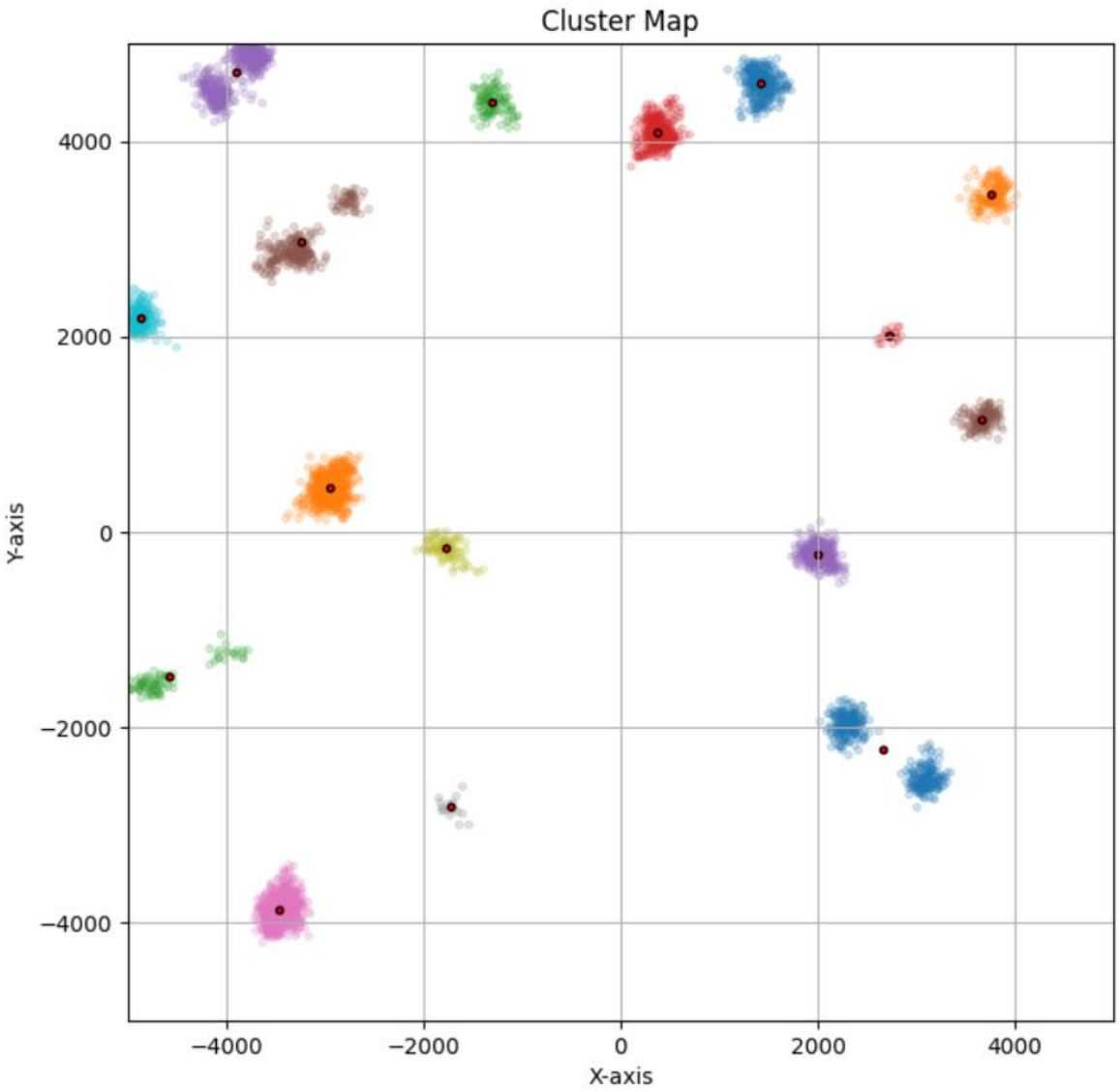
Cluster mapa pre 20020 bodov s použitím centroidov



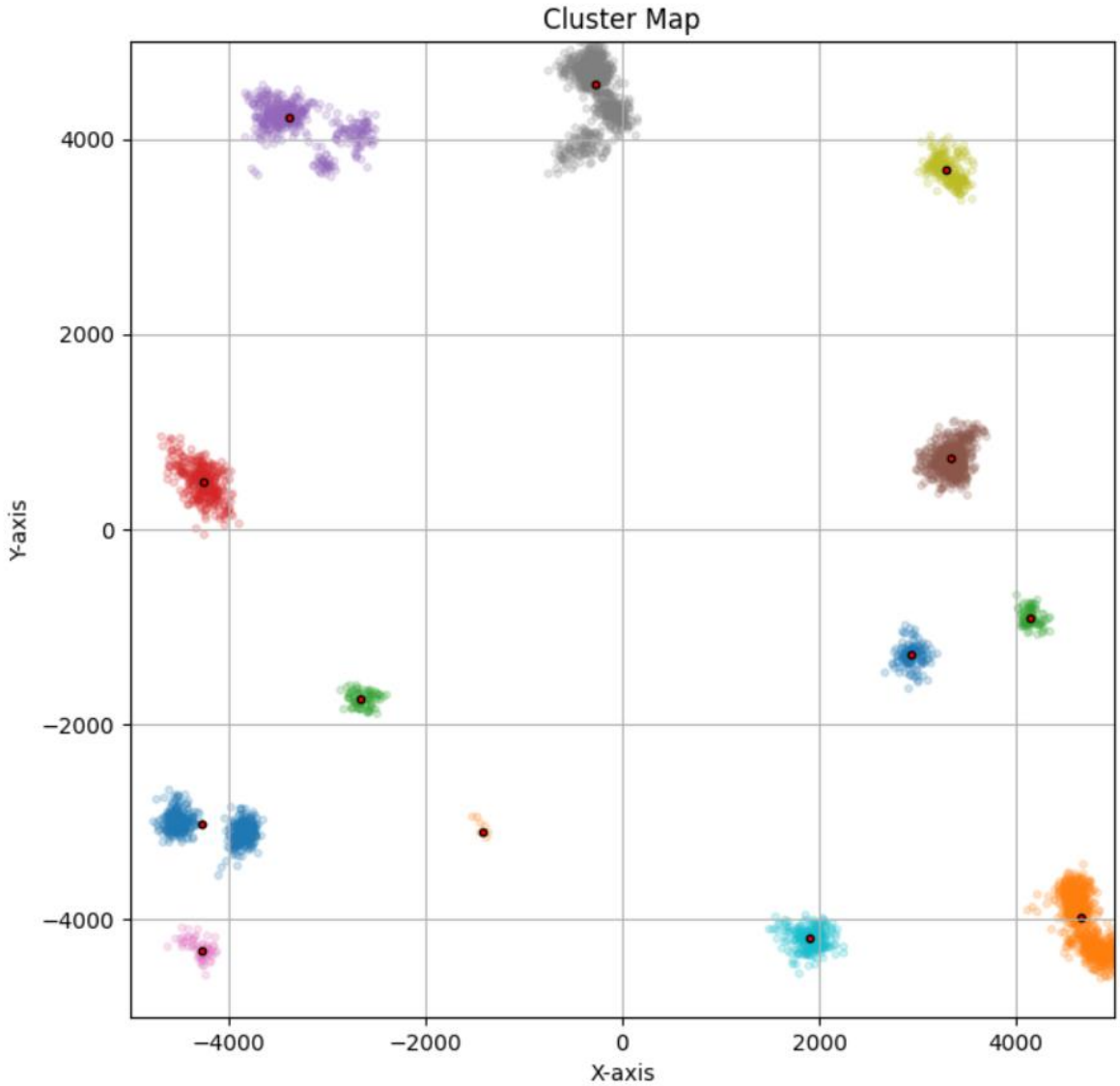
Cluster mapa pre 20020 bodov s použitím medoidov



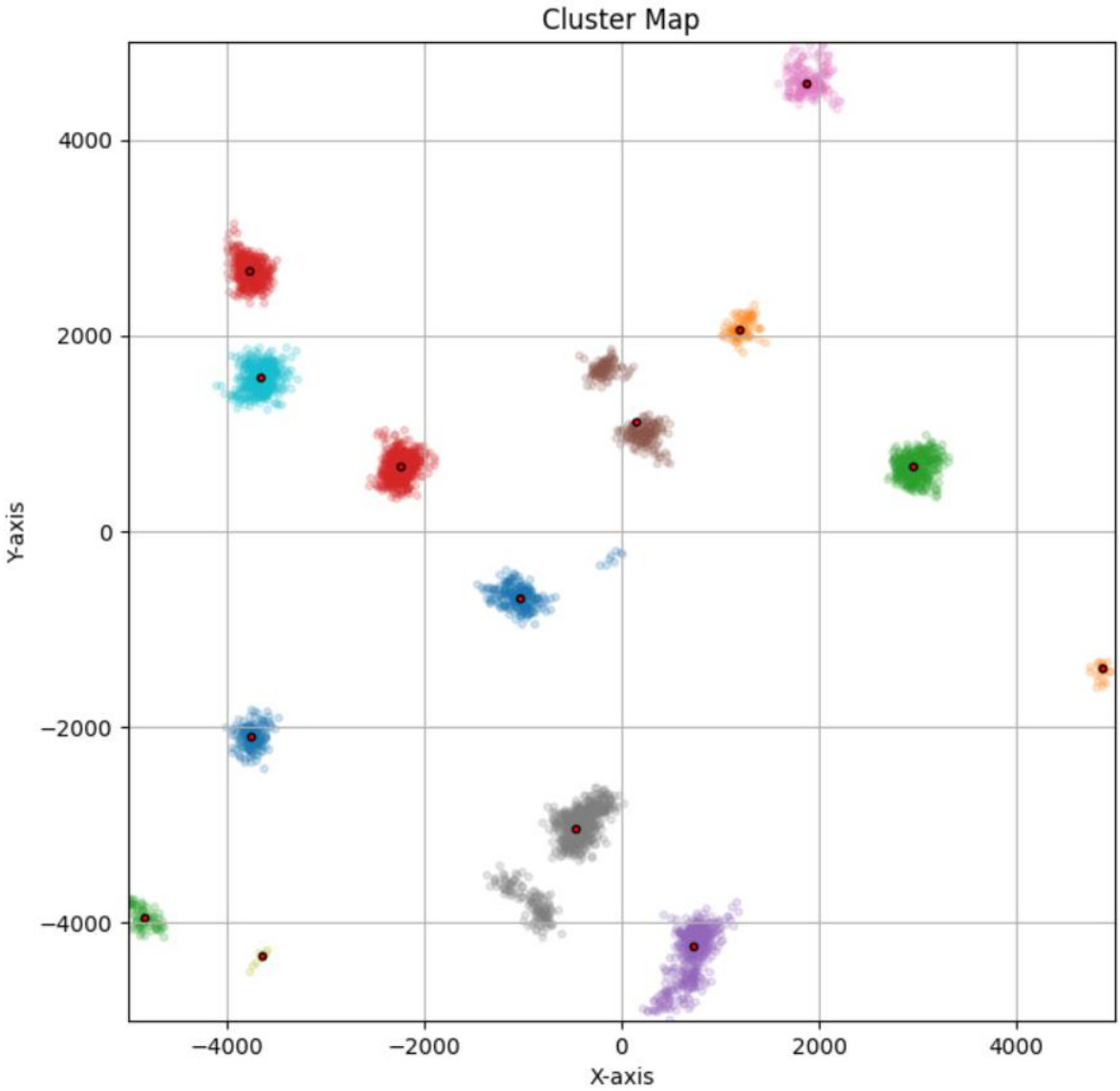
Ďalší pokus pre 4000 bodov s použitím centroidov



Ďalší pokus pre 4000 bodov s použitím centroidov



Ďalší pokus pre 4000 bodov s použitím medoidov



Ďalší pokus pre 4000 bodov s použitím medoidov

Porovnanie algoritmov:

Z uvedených pokusov je vidieť rozdiel medzi medoidami a centroidami. Centroidy sa nachádzajú čo najbližšie k stredu klastru, a sú náchylnejšie na posun kvôli vzdialeným skupinám bodov. Predstavujú reálny stred klastra, zatiaľ čo medoidy sú menej náchylné k posunu, a častejšie predstavujú stred najväčšej skupiny bodov v danom klastri, s menším ohľadom na odľahlé menšie skupiny bodov, ktoré stále patria ku klastru.